

Implementação Apriori para Modelo de Recomendação de Usuários

Autores: João Vitor da Conceição de Almeida
Deivid Souza dos Santos Oliveira Flávio
André Gomes Neto

Agenda

- O objetivo dessa apresentação é apresentar o resultado do desenvolvimento do modelo de recomendação de usuários, implementando Apriori

1. Contextualização:

Apresentação do objetivo Principal

2. Funcionalidades :

Lista das saídas e utilidades práticas que o sistema entregará:
geração de itemsets frequentes; cálculo de suporte, confiança e lift;
lista ordenada de regras acionáveis;

3. Implementação das Funcionalidades:

Exemplificação de implementações das funcionalidades a serem atendidas

4. Resultados:

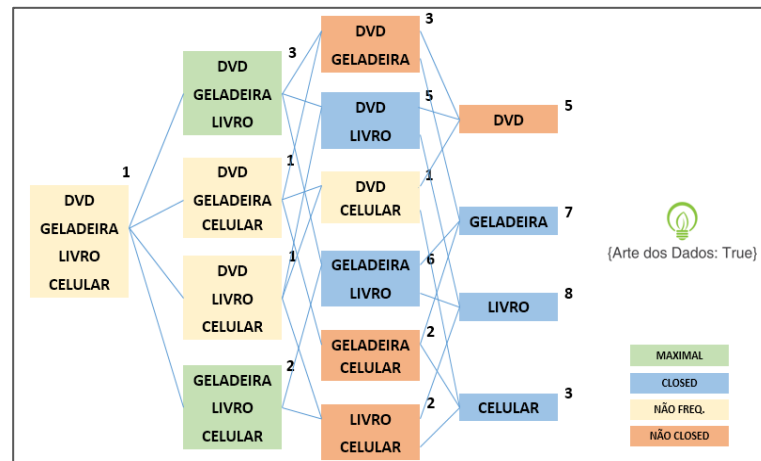
Validação dos métodos implementados

5. Considerações finais:

Desafios enfrentados e comparativo

Contextualização

- Algoritmo de aprendizado não supervisionado usado para encontrar conjuntos de itens frequentes (itemsets) e gerar regras de associação.
- Desenvolvimento de um modelo de recomendação para os usuários
- Extrair padrões de consumo e gerar insights para combos de produtos no segmento de vestuário
- Identificação de padrões frequentes nos dados



Funcionalidades



- **Apriori: (Estrutura)**

Aplicação	Função
Pré-processamento dos dados	• Ler o dataset de transações. • Tratar os itens (ex.: remover marcas das descrições, padronizar nomes). • Transformar em uma estrutura adequada (lista de transações ou matriz binária).
Geração de itemsets frequentes	• Implementar a função para gerar candidatos (Ck). • Calcular suporte de cada conjunto. • Filtrar apenas os que atingem o suporte mínimo (Lk). • Iterar até não haver mais conjuntos frequentes.
Geração de regras de associação	• A partir dos itemsets frequentes, gerar regras do tipo $A \rightarrow B$. • Calcular confiança e lift. • Filtrar regras que atendam aos thresholds definidos.

Funcionalidade vs Implementação



- Pré-Processamentos dos Dados

“Entrada”

```
[  
"Camiseta Nike Infantil Moda", "Shorts Adidas Adulto",  
"Calças Jeans Puma", "Tênis Mic 42", "Blusa Star Unissex",  
"Calças Jeans Nike Moda"  
]
```

“Saida”

```
[  
'camisa', 'bermuda', 'calca jeans', 'tênis 42', 'camisa',  
'calca jeans'  
]
```

```
def normalizar_semantica(item: str) -> str: 1 usage 吕 Joao *  
  
    tokens = item.split()  
    tokens_filtrados = []  
    for tok in tokens:  
  
        if tok in MARCAS_COMUNS:  
            continue  
  
        if tok in PALAVRAS_IRRELEVANTES:  
            continue  
  
        if tok in SINONIMOS:  
            tok = SINONIMOS[tok]  
        |  
        if tok.endswith("s") and len(tok) > 3:  
            tok = tok[:-1]  
            tokens_filtrados.append(tok)  
    return " ".join(tokens_filtrados).strip()
```

Funcionalidade vs Implementação



- Pré-Processamentos de dados

"Entrada"

```
[  
'Camiseta Nike Infantil Moda',  
'Shorts Adidas Adulto', 'Shorts Adidas Adulto'] –  
'Blusa Star Unissex', 'Calças Jeans Puma'] –  
'Tênis Mic 42', 'Conjunto Infantil Liso'] –  
'Camiseta Selfie', 'Shorts Puma', 'Tênis Nike 39'] –  
'Calças Jeans', 'Blusa Moda', 'Tênis 40'  
]
```

"Saida"

```
[  
'camisa', 'bermuda'] – ['camisa', 'calça jean'] –  
['tenis'] – ['camisa', 'bermuda', 'tenis'] – ['calça jeans',  
'camisa', 'tenis'  
]
```

```
def preprocessar_transacoes(transacoes_raw, remover_numeros: bool = REMOVER_NUMEROS): 1 usage  
  
    transacoes_norm = []  
    for trans in transacoes_raw:  
        itens = []  
        for it in trans:  
            it2 = normalizar_item(it, remover_numeros)  
            if it2:  
                itens.append(it2)  
  
        seen = set()  
        itens_unicos = []  
        for it in itens:  
            if it not in seen:  
                seen.add(it)  
                itens_unicos.append(it)  
        if itens_unicos:  
            transacoes_norm.append(itens_unicos)  
    transacoes_sets = [set(t) for t in transacoes_norm]  
    return transacoes_norm, transacoes_sets
```

Funcionalidade vs Implementação



- Geração de item sets frequentes

“Entrada”

candidatos =

```
[ {"camisa"}, {"tenis"},  
  {"camisa", "tenis"}, {"bermuda", "tenis"},  
  {"calca jean", "camisa"}  
]
```

“Saida”

{'camisa'}: 4 – {'tenis'}: 3

{'camisa', 'tenis'}: 2 –

{'bermuda', 'tenis'}: 1 –

{'calca jean', 'camisa'}: 2

```
def calcular_suporte(transacoes_sets, candidatos): 2 usages
```

```
    cont = Counter()
```

```
    for t in transacoes_sets:
```

```
        for c in candidatos:
```

```
            if c.issubset(t):
```

```
                cont[c] += 1
```

```
    return cont
```

Funcionalidade vs Implementação



- Geração de item sets frequentes

“Entrada”

```
[{"camisa", "bermuda"}, {"camisa", "calca jean"}, {"tenis"}, {"camisa", "bermuda", "tenis"}, {"calca jean", "camisa", "tenis"}]
```

“Saida”

Nível 1:

- {'camisa'}: 4
- {'tenis'}: 3
- {'bermuda'}: 2
- {'calca jean'}: 2

Nível 2:

- {'camisa', 'tenis'}: 2
- {'camisa', 'bermuda'}: 2
- {'camisa', 'calca jean'}: 2

```
def apriori(transacoes_sets, min_support=0.05):  @ Deivid Souza *
```

```
    total = len(transacoes_sets)
```

```
    # L1
```

```
    cont1 = calcular_suporte(transacoes_sets, [{i} for tx in transacoes_sets for i in tx])
```

```
    L1 = filtrar_por_suporte(cont1, total, min_support)
```

```
    L = {1: L1}
```

```
    k = 1
```

```
    while L[k]:
```

```
        candidatos = gerar_candidatos([frozenset(x) for x in L[k].keys()])
```

```
        cont_k = calcular_suporte(transacoes_sets, candidatos)
```

```
        Lk = filtrar_por_suporte(cont_k, total, min_support)
```

```
        if not Lk:
```

```
            break
```

```
        k += 1
```

```
        L[k] = Lk
```

```
    return L
```


Funcionalidade vs Implementação



- Regras de Associação

“Entrada”

```
[{"camisa", "bermuda"}, {"camisa", "calca jean"}, {"tenis"}, {"camisa", "bermuda", "tenis"}, {"calca jean", "camisa", "tenis"}]
```

“Saida”

Itemset frequente: {'camisa', 'tenis', 'calca jean'}

Regras geradas:

- {'camisa'} → {'tenis', 'calca jean'} -
- {'tenis'} → {'camisa', 'calca jean'} -
- {'calca jean'} → {'camisa', 'tenis'} -
- {'camisa', 'tenis'} → {'calca jean'} -
- {'camisa', 'calca jean'} → {'tenis'} -
- {'tenis', 'calca jean'} → {'camisa'}

```
def generate_candidate_rules(itemset: FrozenSet[str]) -> List[Tuple[Set[str], Set[str]]]:  
  
    rules = []  
    items = list(itemset)  
    for i in range(1, len(items)):  
        for antecedent in combinations(items, i):  
            antecedent = set(antecedent)  
            consequent = set(items) - antecedent  
            if consequent:  
                rules.append((antecedent, consequent))  
  
    return rules
```

Funcionalidade vs Implementação



- Regras de Associação

“Entrada”

```
simuladossupport_data = {  
    frozenset(["camisa"]): 0.8,  
    frozenset(["tenis"]): 0.6,  
    frozenset(["camisa", "tenis"]): 0.4,  
        frozenset(["calca jean"]): 0.4,  
    frozenset(["camisa", "calca jean"]): 0.3,  
    frozenset(["camisa", "tenis", "calca jean"]): 0.2}
```

tenisantecedent = {"camisa"}

consequent = {"tenis"}

“Saida”

Confiança da regra {'camisa'} $\Rightarrow$ {'tenis'}: 0.50

```
def calculate_confidence(support_data: Dict[Frozenset[str], float],  
                        antecedent: Set[str],  
                        consequent: Set[str]) -> float:  
  
    union = frozenset(antecedent | consequent)  
    return support_data[union] / support_data[frozenset(antecedent)]
```

Resultados



- Todos os Testes passaram

Considerações finais



- A efetividade do modelo Apriori está diretamente ligada à qualidade e consistência dos dados de entrada.
- O pré-processamento técnico e semântico foi essencial para reduzir fragmentação de itens, aumentar suporte e gerar regras mais confiáveis e interpretáveis.
- A implementação foi realizada em Python puro, com foco em modularidade, clareza e reprodutibilidade, sem dependência de bibliotecas externas.
- O pipeline desenvolvido serve como base sólida para sistemas de recomendação personalizados, com potencial de aplicação em ambientes reais de varejo.
- A abordagem adotada permite justificar, academicamente, o ganho de cobertura por redução dimensional semântica, fortalecendo a análise estatística e a tomada de decisão.
- O projeto demonstra que a inteligência está no pré-processamento, e não apenas no algoritmo — uma lição valiosa para aplicações em ciência de dados.